

Laboratorio di Ingegneria Informatica 2026

Lorenzo Castelli
2115498

Ludovica Fiocchetta
2103669

Alessandro Genovese
2108670

1 Introduzione

Il progetto illustra una piattaforma a microservizi progettata per l'estrazione (parsing), l'archiviazione e la valutazione automatizzata di documenti web. L'obiettivo centrale è ricavare testo pulito tramite **Crawl4AI** e misurare l'accuratezza dei parser rispetto a dataset di riferimento (**gold standard**), impiegando sia metriche sintattiche deterministiche sia tecniche di **Intelligenza Artificiale (LLM-as-a-Judge)**. I domini analizzati sono:

- en.wikipedia.org
- www.viaggi-usa.it
- www.cbsnews.com
- www.cnbc.com

2 Organizzazione del codice

Il sistema separa logicamente le operazioni di backend (estrazione, valutazione e persistenza) dall'interfaccia utente (frontend). L'intera infrastruttura è containerizzata tramite **Docker** (`docker-compose.yml`) per assicurare scalabilità, portabilità e totale riproducibilità dell'ambiente di esecuzione.

2.1 Backend

Il backend, esposto sulla porta 8003, presenta un'architettura modulare così strutturata:

- **Server API (`server.py`)**: Basato su **FastAPI** e **Pydantic**, gestisce gli endpoint REST, la validazione rigorosa dei dati e l'instradamento dinamico delle richieste ai parser in base al dominio.
- **Pipeline di Parsing (`crawler_*.py`)**: Moduli specifici per dominio basati su **Crawl4AI**, che operano in tre fasi:
 1. *Acquisizione*: Tramite browser *headless* (con bypass della cache per dati in tempo reale) o estrazione statica da stringa HTML.

2. *Pulizia DOM*: Iniezione **JavaScript** per rimuovere dinamicamente elementi di disturbo (es. banner, sidebar).

3. *Post-processing*: Filtraggio del Markdown tramite **Regex** per eliminare tag residui e metadati, ottenendo testo pulito.

- **Valutazione (`accuracy.py`)**: Servizio indipendente che calcola l'accuratezza del crawler misurando la sovrapposizione dei token (previa normalizzazione testuale) tra il testo estratto e il *gold standard*.
- **Utility (`init_db.py`, `remove_markdown.py`)**: Script ausiliari per l'inizializzazione del database e la rimozione della formattazione Markdown.

2.2 Frontend

L'interfaccia utente, esposta sulla porta 8004, fornisce un accesso intuitivo al sistema ed è composta dai seguenti moduli:

- **frontend.py**: Sviluppato con **FastAPI**, agisce come un'interfaccia tra l'utente e il backend. Utilizza il client asincrono **httpx** per comunicare internamente con i servizi del backend, recuperando dinamicamente la lista dei domini supportati, gli URL presenti nei Gold Standard e lo stato dei componenti (Backend, MariaDB, Ollama) in tempo reale, gestendo così l'intero ciclo richiesta risposta.
- **templates/**: Utilizza il motore **Jinja2** per il rendering HTML dinamico. Struttura l'interfaccia in aree specializzate: una home di stato, un'interfaccia di parsing a tre colonne (Row HTML, Gold Standard, testo parsato) con metriche, un pannello di gestione dei Gold Standard, e una pagina analitica con statistiche complessive di ciascun dominio e giudizio dell'LLM.
- **static/**: Contiene i fogli di stile **CSS** che definiscono l'estetica dell'applicazione.

2.3 Dati e Configurazione

- **domains.json**: File di configurazione che elenca i domini di riferimento.
- **gs_data/**: Contiene, in formato JSON, i Gold Standard necessari per il calcolo delle metriche di accuratezza.

3 Database

L'architettura integra **MariaDB** per la persistenza dei dati e il *caching* locale, permettendo di riutilizzare l'HTML già salvato per velocizzare le valutazioni successive senza ripetere il download dalla rete.

3.1 Tabelle relazionali

Le tabelle implementate (con chiave primaria sottolineata) sono:

- **web_resources** (url, domain, title, html_text, created_at): Memorizza l'HTML originale, il titolo e il dominio delle pagine web richieste.
- **gold_standard** (url, gold_text, created_at): Conserva i testi di riferimento.
- **parsed_pages** (url, parsed_text, created_at): Salva il testo pulito ed estratto dal crawler.
- **evaluation_results** (url, token_level_metrics, LLM_judgments, evaluated_at): Archivia le metriche di base (Sez. 5) e avanzate (Sez. 4) calcolate per l'URL target.

3.2 Gestione Tabelle

Al primo avvio, il database si inizializza popolando automaticamente le tabelle dai file in /gold_standard (con le metriche in evaluation_results impostate a NULL). Successivamente, le tabelle vengono aggiornate dinamicamente durante le richieste di parsing e valutazione. Sono inoltre presenti endpoint specifici per l'eliminazione manuale delle entry.

4 Valutazione dei Judge

L'implementazione del paradigma *LLM-as-a-Judge* sfrutta il modello llama3.2 (eseguito localmente tramite **Ollama**) per valutare l'accuratezza semantica del testo estratto rispetto al *Gold Standard*.

L'ottimizzazione del processo di valutazione si fonda sui seguenti criteri:

- **Gestione del contesto**: Per garantire stabilità ed efficienza, l'input (sia estratto che di riferimento) è rigorosamente limitato a 1000 caratteri.
- **Ingegneria del Prompt**: Il modello è istruito per agire da valutatore esperto, assegnando un punteggio intero da 1 (completamente errato) a 5 (perfettamente accurato).
- **Generazione Deterministica**: Per azzerare la stocasticità dei LLM e garantire replicabilità, la temperatura è impostata a 0.0. Inoltre, è imposta la modalità `format="json"` per forzare l'output di un oggetto strutturato contenente solo le chiavi `judge_score` e `judge_feedback`.

5 Valutazione parser

La qualità dell'estrazione è misurata confrontando l'output dei parser con i dati di riferimento (Gold Standard) situati in `gs_data/`.

5.1 Metodologia di valutazione

Il modulo `accuracy.py` calcola le metriche di base *token-level* (Precision, Recall e F1-score) affiancandole a tre metriche avanzate (`x_eval`) per un'analisi più rigorosa:

- **Jaccard Similarity**: Definito come il rapporto tra l'intersezione e l'unione degli insiemi testuali ($|A \cap B| / |A \cup B|$), offre un indicatore severo della sovrapposizione del vocabolario, penalizzando fortemente l'aggiunta di rumore o la perdita di dati.
- **Bigram Overlap**: Misura la sovrapposizione di bigrammi (coppie di parole consecutive) per verificare e premiare la conservazione della struttura sintattica locale e dell'ordine logico delle frasi estratte.
- **Cosine Similarity**: Valuta l'affinità semantica calcolando il coseno dell'angolo tra i vettori *TF-IDF* dei testi. È ideale per confermare il focus sui concetti chiave, ignorando le minime variazioni testuali o la presenza di *stop-words*.

5.2 Risultati ottenuti

Nelle Tabelle sono riportati i risultati medi ottenuti per ciascun dominio target sull'intero Gold Standard, generati tramite l'endpoint `/full_gs_eval`. I valori permettono di quantificare oggettivamente l'efficacia dei parser.

Dominio	Precision	Recall	F1-score
en.wikipedia.org	0.9979	0.9989	0.9984
www.cbsnews.com	1.0000	1.0000	1.0000
www.cnbc.com	0.9995	0.9985	0.9990
www.viaggi-usa.it	0.9919	0.9989	0.9953

Dominio	Jaccard S.	Bigram O.	Cosine S.
en.wikipedia.org	0.9969	0.9983	0.9998
www.cbsnews.com	1.0000	1.0000	0.9999
www.cnbc.com	0.9981	0.9992	0.9986
www.viaggi-usa.it	0.9909	0.9976	0.9988

Table 1: Metriche di accuratezza Token-level (sopra) e metriche avanzate (sotto) ottenute dai parser per dominio.

Dominio	LLM Judge Score
en.wikipedia.org	4.4
www.cbsnews.com	4.5
www.cnbc.com	4.6
www.viaggi-usa.it	5.0

Table 2: Valutazione semantica media assegnata dall’LLM Judge (llama3.2) per ciascun dominio analizzato.